



ПЛАТФОРМА НЕКСУС

AGILE (SCRUM) против стандартного ТЗ

Версия: 1.0

Статус: ТЗ vs AGILE (SCRUM)

ОГЛАВЛЕНИЕ

ПРИРОДА ЗАДАЧИ: ФРОНТ-ПРОЕКТ, НЕ ПРОИЗВОДСТВО	2
ПОЧЕМУ КЛАССИЧЕСКОЕ ТЗ НЕ РАБОТАЕТ ДЛЯ НЕКСУС	2
2.1 ПРОБЛЕМА 1: СЛИШКОМ БЫСТРО МЕНЯЮТСЯ ТРЕБОВАНИЯ	2
2.2 ПРОБЛЕМА 2: НЕВОЗМОЖНО ПРЕДУСМОТРЕТЬ ВСЕ ДЕТАЛИ	2
КРИТЕРИИ ПРИЕМКИ + СРОКИ	3
3.1 ПРИНЦИП 1: КРИТЕРИИ ПРИЕМКИ (ACCEPTANCE CRITERIA)	3
3.2 ПРИНЦИП 2: СРОКИ ВМЕСТО ПОДРОБНЫХ СПЕЦИФИКАЦИЙ	3
3.3 ФОРМАТ КРИТЕРИЕВ ПРИЕМКИ: GIVEN-WHEN-THEN	4
КАК РАБОТАЕТ ПРОЦЕСС НЕКСУС	4
4.1 ФАЗА ПЛАНИРОВАНИЯ (ВМЕСТО ТЗ)	4
4.2 ФАЗА РАЗРАБОТКИ (AGILE СПРИНТЫ)	4
4.3 ФАЗА ПРИЕМКИ (ACCEPTANCE TESTING)	5
СРАВНЕНИЕ: ТЗ VS КРИТЕРИИ ПРИЕМКИ	5
МЕТОДОЛОГИЯ: ACCEPTANCE TEST DRIVEN DEVELOPMENT (ATDD)	5
ИТОГОВЫЙ ОТВЕТ: ПОЧЕМУ КРИТЕРИИ, А НЕ ТЗ?	6
Причина 1: НЕКСУС — инновационный проект	6
Причина 2: Законодательство меняется	6
Причина 3: Критерии конкретнее, чем ТЗ	6
Причина 4: Быстрее разработка	6
Причина 5: Больше контроля	6

Причина 6: Меньше возвратов	6
ФОРМУЛА НЕКСУС	6
ВЫВОД	7

ПРИРОДА ЗАДАЧИ: ФРОНТ-ПРОЕКТ, НЕ ПРОИЗВОДСТВО

ТЗ (Техническое задание) подходит для:

- **Известных задач:** Копирование действующих систем и проектов Lender Invest в России
- **Тиражирования:** Разработка 10-го платежного API
- **Стандартных процессов:** Типичный CRUD-приложение
- **Предсказуемых результатов:** "Сделать ровно это и ничего больше"

НЕКСУС — ЭТО ИННОВАЦИОННЫЙ ПРОЕКТ, где:

- **Неизвестные требования:** никто не собирал инвестиции в блокчейне с ЦБ РФ интеграцией
- **Нужны итерации:** понимаем структуру по ходу разработки
- **Меняется законодательство:** законодательная база постоянно развивается
- **Уникальная архитектура:** Собственный блокчейн ГАНИМЕД, биржа СФОРДЕКС, цифровой депозитарий, маркетплейс

ТЗ БУДЕТ УСТАРЕВАТЬ ЕЩЕ ДО НАПИСАНИЯ ПЕРВОЙ СТРОКИ КОДА.

ПОЧЕМУ КЛАССИЧЕСКОЕ ТЗ НЕ РАБОТАЕТ ДЛЯ НЕКСУС

2.1 ПРОБЛЕМА 1: СЛИШКОМ БЫСТРО МЕНЯЮТСЯ ТРЕБОВАНИЯ

Сценарий с классическим ТЗ:

Декабрь 2025: Написано ТЗ (150 страниц)

- └ Раздел 1: Архитектура ГАНИМЕД
- └ Раздел 2: API биржи
- └ Раздел 3: Интеграция с ЦБ РФ
- └ Раздел 4: KYC/AML модули

Январь 2026: Разработка началась

- └ Неделя 1: ЦБ РФ уточнил требования
- └ Неделя 2: ТЗ нужно переписать (1 неделя потеряна)
- └ Неделя 3: Понял, что архитектура неправильная
- └ Неделя 4: Переделка ТЗ (2 недели потеряны)

Март 2026: Только половина спринта выполнена

ПРОБЛЕМА: ТЗ охватывает множество деталей, которые при разработке оказываются неправильными.

2.2 ПРОБЛЕМА 2: НЕВОЗМОЖНО ПРЕДУСМОТРЕТЬ ВСЕ ДЕТАЛИ

Что мы НЕ ЗНАЕМ на этапе написания ТЗ:

1. Блокчейн ГАНИМЕД:

- Какой точно консенсус? (PoA и PoS комбинация)
- Как именно интегрировать с ЦБ РФ? (API не стандартизирован)

- Какой TPS нужен? (Может быть, 5,000 хватит, может, нужно 20,000)
- Как решать конфликты узлов?
- 2. **ВЫВОД:** нужно **разработать первую версию**, увидеть проблемы, переделать.
- 3. **Интеграция с госсистемами:**
 - ГосУслуги может измениться (они обновляют API часто)
 - ЦБ РФ может добавить новые требования
 - ФНС может требовать другой формат данных
- 4. **ВЫВОД:** нельзя заранее знать точные спецификации.
- 5. **AI-ассистент:**
 - Какие параметры анализировать? (50+?)
 - Какие ML-модели использовать?
 - Как обучать на российских данных?
- 6. **ВЫВОД:** нужно **экспериментировать** с моделями.

КРИТЕРИИ ПРИЕМКИ + СРОКИ

3.1 ПРИНЦИП 1: КРИТЕРИИ ПРИЕМКИ (ACCEPTANCE CRITERIA)

Определение:

Acceptance Criteria = **условия, при выполнении которых задача считается выполненной**

Пример критерия приемки (вместо ТЗ):

ЗАДАЧА: Реализовать блокчейн ГАНИМЕД

КРИТЕРИИ ПРИЕМКИ (вместо 120 страниц ТЗ):

- ☒ Нода запускается и синхронизируется за <5 минут
- ☒ TPS >1,000 (может быть доказано нагрузочным тестом)
- ☒ Комиссия за транзакцию <\$0.01
- ☒ Консенсус стабилен на 5 нодах неделю без перезагрузок
- ☒ Возможна интеграция с TRON через мост
- ☒ Блоки подписаны ГОСТ 34.10-2012 (для соответствия РФ)

ПРЕИМУЩЕСТВА:

- **Короче:** 20 строк вместо 120 страниц
- **Измеримо:** Либо работает TPS >1,000, либо нет
- **Гибко:** как реализовать — **решает разработчик**
- **Тестируемо:** Каждый критерий можно проверить автоматически

3.2 ПРИНЦИП 2: СРОКИ ВМЕСТО ПОДРОБНЫХ СПЕЦИФИКАЦИЙ

Традиционный подход (ТЗ):

ТЗ: 150 страниц подробных спецификаций

- └ Разработчик следует спецификации
- └ Если не подходит — нужны согласования
- └ Итерация длится 1-2 месяца

Подход НЕКСУС (критерии + сроки):

Критерий: "TPS >1,000"

Сроки: "Месяц 4 (апрель 2026)"

- └ Разработчик решает как достичь критерия
- └ Если не получается — обсуждаем вариант
- └ Итерация длится 2 недели

└ Можем быстро переделать

ПРЕИМУЩЕСТВО: Гибкость. Если первый подход не работает, пробуем второй.

3.3 ФОРМАТ КРИТЕРИЕВ ПРИЕМКИ: GIVEN-WHEN-THEN

Вместо ТЗ пишем:

КРИТЕРИИ ПРИЕМКИ: Интеграция с ЦБ РФ

Given: Инициатор проекта загрузил ЦФА контракт

When: Платформа отправляет данные в ЦБ РФ API

Then:

- ☒ Регистрация прошла успешно
- ☒ Получен номер в реестре ЦБ
- ☒ Данные синхронизированы за <5 минут
- ☒ Возможна повторная регистрация (повторы)

Это гораздо понятнее, чем:

ТЗ Раздел 3.2.1: "Система должна обеспечивать синхронизацию с API Центрального банка Российской Федерации для регистрации цифровых финансовых активов в соответствии с требованиями Положения ЦБ РФ №104-П и 115-ФЗ..."

КАК РАБОТАЕТ ПРОЦЕСС НЕКСУС

4.1 ФАЗА ПЛАНИРОВАНИЯ (ВМЕСТО ТЗ)

МЕСЯЦ 4 (АПРЕЛЬ 2026): Блокчейн и токенизация

└ Критерий 14: Блокчейн ГАНИМЕД

- └ ☒ Нода работает (TPS >1,000)
- └ ☒ 5 нод сети стабильны 7 дней
- └ ☒ Консенсус PoA + RAFT
- └ СРОКИ: 20 дней

└ Критерий 6: Токенизация проектов

- └ ☒ Смарт-контракт GND-st1 работает
- └ ☒ Tokens создаются за <30 сек
- └ СРОКИ: 12 дней

└ Критерий 4.4: Аудит смарт-контрактов

- └ ☒ Профессиональный аудит пройден
- └ ☒ 0 критичных багов
- └ СРОКИ: 15 дней (параллельно с разработкой)

4.2 ФАЗА РАЗРАБОТКИ (AGILE СПРИНТЫ)

Неделя 1-2: Backend разработчики читают критерии

- └ Думают: "Как мы достигнем TPS >1,000?"
- └ Обсуждают варианты с СТО
- └ Начинают экспериментировать (прототипирование)

Неделя 3-4: Первая итерация

- Разработана базовая версия ГАНИМЕД
- TPS = 500 (не хватает)
- Обсуждение: "Нужно оптимизировать консенсус"
- Идет итерация

Неделя 4-5: Вторая итерация

- Оптимизирован RAFT консенсус
- TPS = 2,000 (превышает критерий)
- ☒ Критерий "TPS >1,000" выполнен
- Переходим к интеграции

Неделя 5-6: Тестирование и интеграция

- QA тестирует 5 нод сети
- ☒ Сеть стабильна 7 дней
- ГОТОВО!

КЛЮЧ: Разработчики **решают, как**, а не следуют спецификации.

4.3 ФАЗА ПРИЕМКИ (ACCEPTANCE TESTING)

КРИТЕРИЙ: TPS >1,000

ПРОВЕРКА (автоматизированный тест):

1. Запустить 100,000 транзакций
2. Измерить время выполнения
3. Вычислить TPS = транзакции / время
4. Если TPS >= 1,000 → ☒ PASS
Если TPS < 1,000 → **✗** FAIL (возврат на доработку)

Просто, объективно, автоматизируемо.

СРАВНЕНИЕ: ТЗ VS КРИТЕРИИ ПРИЕМКИ

ХАРАКТЕРИСТИКА	ТРАДИЦИОННОЕ ТЗ	КРИТЕРИИ ПРИЕМКИ + СРОКИ
ОБЪЕМ	30-150 страниц	20-35 пунктов
ПОНЯТНОСТЬ	✗ Часто неясно	☒ Четко и измеримо
ГИБКОСТЬ	✗ Меняется долго	☒ Быстро адаптируется
ВРЕМЯ СОГЛАСОВАНИЯ	🕒 2-4 недели	☒ 2-3 дня
ТЕСТИРУЕМОСТЬ	⚠️ Сложно	☒ Легко автоматизировать
ИЗМЕНЕНИЯ	✗ 30-40% переписывается	☒ 5-10% уточняются
СКОРОСТЬ РАЗРАБОТКИ	🕒 Медленнее (ждем четкого ТЗ)	☒ Быстрее (сразу работают)
КОНТАКТ С БИЗНЕСОМ	🕒 Редко	☒ Постоянно (каждый спринт)
ВОЗВРАЩЕНИЯ НА ДОРАБОТКУ	✗ Частые (неправильное ТЗ)	☒ Редкие (четкие критерии)

МЕТОДОЛОГИЯ: ACCEPTANCE TEST DRIVEN DEVELOPMENT (ATDD)

НЕКСУС использует **ATDD**, где:

1. **Сначала пишут критерии приемки** (не код)

- ☒ TPS >1,000
- ☒ Сеть работает стабильно 7 дней
- ☒ Комиссия <\$0.01

2. **Потом пишут автоматические тесты** для этих критериев

```
def test_throughput():
    throughput = measure_tps()
    assert throughput > 1000, f"TPS = {throughput}"
```

3. **Потом разработчик пишет код**, чтобы тесты прошли

4. **Приемка = тесты прошли** ☒

Преимущество ATDD:

- ☒ Четкое определение "готово"
- ☒ Нет субъективности
- ☒ Быстро видно прогресс (сколько тестов прошло)

ИТОГОВЫЙ ОТВЕТ: ПОЧЕМУ КРИТЕРИИ, А НЕ ТЗ?

Причина 1: НЕКСУС — инновационный проект

- ☒ Нельзя заранее знать все детали
- ☒ Нужны итерации и эксперименты
- ☒ Критерии позволяют итерировать
- ☒ ТЗ был бы устаревшим перед стартом

Причина 2: Законодательство меняется

- ☒ 115-ФЗ развивается (новые требования)
- ☒ ЦБ РФ обновляет API
- ☒ Критерии адаптируются за часы
- ☒ ТЗ требует переписывания за недели

Причина 3: Критерии конкретнее, чем ТЗ

- ☒ ТЗ: "Система должна быть быстрой" (неясно)
- ☒ Критерии: "TPS >1,000" (четко)
- ☒ Критерии: "Комиссия <\$0.01" (измеримо)

Причина 4: Быстрее разработка

- ☒ ТЗ: Согласование 2-4 недели, потом разработка
- ☒ Критерии: Согласование 2-3 дня, сразу разработка

Причина 5: Больше контроля

- ☒ ТЗ: Разработчик следует спецификации слепо
- ☒ Критерии: Разработчик решает КАК, бизнес решает ЧТО

Причина 6: Меньше возвратов

- ☒ ТЗ: 30-40% переписывается в процессе
- ☒ Критерии: 5-10% уточняются в процессе

ФОРМУЛА НЕКСУС

КРИТЕРИИ ПРИЕМКИ + СРОКИ = (ГИБКОСТЬ AGILE) + (ОТВЕТСТВЕННОСТЬ ТЗ)

Вместо: ТЗ (150 стр) → Разработка → Тестирование

Используем: Критерии (20 пунктов) → Разработка & Тестирование параллельно

Результат:

- ☒ 6 месяцев вместо 12-18
- ☒ \$350,000 бюджета
- ☒ 92% готовности MVP
- ☒ Легко адаптироваться к изменениям

ВЫВОД

ТЗ — это хорошо для **известных, стабильных, тиражируемых задач**.

Критерии приемки + сроки — это необходимо для **инновационных, изменчивых, уникальных проектов**

НЕКСУС — не тиражирование, это инновация. Поэтому критерии приемки, а не ТЗ.